

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 hr = pd.read_csv(r"C:\Users\Harikesh\Downloads\HRDataset_v14.csv")
6
7 hr.head()
8
9 hr.info()
10
11 hr.isnull().sum()
12
13 hr.describe()
14
15 hr.drop(columns='DateofTermination', inplace=True)
16
17 # FEATURE ENGINEERING
18
19 hr.columns = hr.columns.str.strip()
20
21 # Create Age from DOB
22
23 hr['DOB'] = pd.to_datetime(hr['DOB'])
24
25 # Calculate Age
26
27 current_year = pd.Timestamp.now().year
28 hr['Age'] = current_year - hr['DOB'].dt.year
29
30 # Create age Groups
31 hr['Age_Group'] = pd.cut(
32     hr['Age'],
33     bins = [-49,30,40,50,],
34     labels=[
35         'Young Employees',
36         'Mid Level Employees',
37         'Senior Employees',
38         #'Experience Employees'
39     ],
40
41     include_lowest=True
42 )
43
44 hr[['DOB', 'Age', 'Age_Group']].head()
45
46 hr['Age_Group'].unique()
47
48 import pandas as pd
49
50 # Convert date columns into datetime format
51 # We use pd.to_datetime() to convert the column into a proper date format so Python
   can perform date calculations correctly.
52
53 #In your dataset, DateofHire is usually stored as text/string, not as an actual
   date.
54 hr['DateofHire'] = pd.to_datetime(hr['DateofHire'])
55
56 # Calculate Total Years in Company
57 # it gives the current date and time pd.Timestamp.now().
58 current_date = pd.Timestamp.now()

```

```

59
60
61 # pd.cut() is used to divide continuous numeric data into groups or categories
    (bins).
62
63 # In your project, it is used to convert employee experience years into meaningful
    categories.
64 hr['Total_Years_in_Company'] = (
65     (current_date - hr['DateofHire']).dt.days / 365
66 ).round(1)
67
68 # Create Tenure Categories
69 hr['Tenure_Category'] = pd.cut(
70     hr['Total_Years_in_Company'],
71     bins=[0, 1, 3, 5, 100],
72     labels=[
73         'New Joiner',
74         '1-3 Years',
75         '3-5 Years',
76         '5+ Years'
77     ],
78     include_lowest=True
79 )
80
81 # Display result
82 hr[['DateofHire', 'Total_Years_in_Company', 'Tenure_Category']].head()
83
84 # Marital Status Group
85
86 hr['MaritalStatusID'].value_counts()
87
88 hr[['MaritalStatusID', 'MaritalDesc']].drop_duplicates()
89
90 hr['Marital_Status_Group'] = hr['MaritalStatusID'].replace({
91     0: 'Single',
92     1: 'Married',
93     2: 'Divorced',
94     3: 'Divorced',
95     4: 'Divorced'
96 })
97
98 hr['Marital_Status_Group'].value_counts()
99
100 hr['PerformanceScore'].value_counts()
101
102 # This Methode are used to combine or create group same of elemenet
103 hr['Performance_Group'] = hr['PerformanceScore'].replace({
104     'Exceeds' : 'High Performance',
105     'Fully' : 'Good Performance',
106     'Needs Improvement' : 'Low Performance',
107     'PIP' : 'Low Performance'
108 })
109
110 hr[['PerformanceScore', 'Performance_Group']]
111
112 hr['Performance_Group'].value_counts()
113
114 hr['EngagementSurvey'].value_counts()
115
116 hr['Engagement_Bands'] = pd.cut(

```

```

117     hr['EngagementSurvey'],
118     bins=[0,2,4,5],
119     labels=[
120         'Low Engagement',
121         'Medium Engagement',
122         'High Engagement'
123     ],
124     include_lowest= True
125 )
126 hr[['Engagement_Bands', 'EngagementSurvey']]
127
128 hr['Engagement_Bands'].value_counts()
129
130 hr['Group_Absence'] = pd.cut(
131     hr['Absences'],
132     bins = [0,5,10,17],
133     labels=[
134         'Low Absence',
135         'Moderate Absenece',
136         'High Absence'
137     ],
138     include_lowest=True
139 )
140
141 hr[['Group_Absence', 'Absences']]
142
143 hr['Group_Absence'].value_counts()
144
145 hr['DaysLateLast30'].value_counts()
146
147 hr['Late_Arrival_Risk'] = pd.cut(
148     hr['DaysLateLast30'],
149     bins=[-1,0,2,10],
150     labels=[
151         'Never Late',
152         'Occasionally Late',
153         'Frequently Late'
154     ]
155 )
156
157 hr[['DaysLateLast30', 'Late_Arrival_Risk']]
158
159 hr['Late_Arrival_Risk'].value_counts()
160
161 # Employment Features
162
163 hr['Termd'].value_counts()
164
165 hr['Employment_Group'] = hr['EmploymentStatus'].replace({
166     'Active' : 'Active',
167     'Voluntarily Terminated' : 'Left Company',
168     'Terminated for Cause' : 'Left Company'
169 })
170
171 hr[['Employment_Group', 'EmploymentStatus']].head()
172
173 hr['Department_Group'] = hr['Department'].replace({
174     'Production' : 'Sales & Marketing',
175     'IT/IS' : 'Technical',
176     'Software Engineering' : 'Technical',

```

```

177     'Admin Offices' : 'Operations',
178     'Executive Office' : 'Operations'
179 })
180
181 hr[['Department_Group', 'Department']].head()
182
183 # Salary Engineering
184
185 hr['Salary'].value_counts()
186
187 hr['Salary'].describe()
188
189 hr['Salary_Band'] = pd.cut(
190     hr['Salary'],
191     bins=[0, 50000, 80000, 220450],
192     labels=[
193         'Low Salary',
194         'Medium Salary',
195         'High Salary'
196     ],
197     include_lowest=True
198 )
199
200 hr[['Salary', 'Salary_Band']]
201
202 # Hiring(Year,Month,Day)
203
204 # Convert DateofHire into Datetime format
205 hr['DateofHire'] = pd.to_datetime(hr['DateofHire'])
206
207
208 # Extract Hiring Year
209
210 hr['Hiring_Years'] = hr['DateofHire'].dt.year
211
212 # Extract Hiring Month
213
214 hr['Hiring_Month'] = hr['DateofHire'].dt.month
215
216 # Extract Hiring month name
217 hr['Hiring_Month_Name'] = hr['DateofHire'].dt.month_name()
218
219 # Extract Hiring Quarter
220
221 hr['Hiring_Quarter'] = hr['DateofHire'].dt.quarter
222
223 hr[[
224     'DateofHire',
225     'Hiring_Years',
226     'Hiring_Month',
227     'Hiring_Month_Name',
228     'Hiring_Quarter'
229 ]]
230
231 df = hr.drop(columns=[
232     'Employee_Name',
233     'EmpID',
234     'Zip',
235     'ManagerName'
236 ])

```

```
237
238
239 ## # EDA and Visualization
240
241 # Employee Attrition Analysis
242
243 import seaborn as sns
244 import matplotlib.pyplot as plt
245
246 sns.countplot(x = 'Employment_Group', data = hr)
247
248 plt.title('Employee Attrition Analysis')
249 plt.xlabel('Employment Status')
250 plt.ylabel('Number of Employees')
251
252 # Rotate labels for better visibility
253 plt.xticks(rotation = 15)
254
255 plt.show()
256
257
258 # Heatmap
259
260 # They are only selected numerical columns
261
262 numeric_df = hr.select_dtypes(include=['number'])
263 corr = numeric_df.corr()
264
265 plt.figure(figsize=(12,8))
266 sns.heatmap(corr,
267             annot=True,
268             fmt='.2f',
269             linewidths=0.5)
270
271
272 # Salary Band
273
274 sns.countplot( x = 'Salary_Band', data=hr)
275 plt.title('Employee Salary')
276 plt.xlabel('Salary')
277 plt.ylabel('Number of Employee')
278
279 plt.show()
280
281 # Salary Distribution
282
283 sns.histplot(hr['Salary'], bins=20)
284
285 plt.title('Salary Distribution')
286 plt.xlabel('Salary')
287 plt.ylabel('Frequency')
288
289 plt.show()
290
291 # Salary Distribution
292
293 sns.boxplot(
294     x = 'Employment_Group',
295     y = 'Salary',
296     data = hr
```

```
297 )
298
299 plt.title('Salary by Employment Status')
300 plt.show()
301
302 # Performance Analysis
303
304 sns.countplot(x = 'Performance_Group',data=hr)
305
306 plt.title('Performance Analysis')
307 plt.xlabel('Performance')
308 plt.ylabel('Number of Employee')
309
310 plt.show()
311
312 # Employee Satisfaction Analysis
313
314 sns.histplot(hr['EmpSatisfaction'], bins=5,kde=True)
315
316 plt.title('Employee Satisfaction Distribution')
317 plt.xlabel('Satisfaction Score')
318 plt.ylabel('Frequency')
319
320 plt.show()
321
322 # Absence Analysis
323
324 sns.boxplot(
325     x = 'Employment_Group',
326     y = 'Absences',
327     data = hr
328 )
329
330
331 plt.title('Absences by Employment Status')
332 plt.show()
333
334 # Late Arrival Analysis
335
336 sns.countplot(x = 'Late_Arrival_Risk',data = hr)
337
338 plt.title('Late Arrival Analysis')
339 plt.xlabel('Arrival')
340
341 plt.ylabel('Number of Empoyees')
342
343 plt.show()
344
345 # Hiring Trend Analysis
346
347 sns.countplot(x='Hiring_Years', data=hr)
348
349 plt.title('Hiring Trend')
350 plt.xlabel('Year')
351 plt.ylabel('Number of Employees')
352
353 plt.xticks(rotation=45)
354
355 plt.show()
356
```

```

357 # Department-wise Salary Analysis
358
359 sns.boxplot(
360     x = 'Department',
361     y = 'Salary',
362     data = hr
363 )
364
365 plt.title('Department wise Salary Analysis')
366 plt.xlabel('Department')
367 plt.ylabel('Salary')
368
369 plt.xticks(rotation = 45)
370
371 plt.show()
372
373 # Age Distribution
374
375 plt.figure(figsize=(12,8))
376
377 # Age Distribution
378 sns.countplot(x='Age_Group', data=hr)
379
380 # Title and labels
381 plt.title('Age Distribution')
382 plt.xlabel('Age Group')
383 plt.ylabel('Number of Employees')
384
385 plt.show()
386
387 # Machine Learning to Predict Attrition
388
389 import pandas as pd
390 from sklearn.model_selection import train_test_split
391 from sklearn.preprocessing import LabelEncoder, StandardScaler
392 from sklearn.linear_model import LogisticRegression
393 from sklearn.metrics import accuracy_score
394
395 # Load Dataset
396 df = pd.read_csv(r"C:\Users\Harikesh\Downloads\HRDataset_v14.csv")
397
398 # Fill Missing Values
399 df = df.fillna(0)
400
401 # Convert String Columns
402 le = LabelEncoder()
403
404 for col in df.select_dtypes(include='object').columns:
405     df[col] = le.fit_transform(df[col].astype(str))
406
407 # Target Column
408 y = df['Termd']
409
410 # Features
411 X = df.drop(columns=['Termd'])
412
413 # Train Test Split
414 X_train, X_test, y_train, y_test = train_test_split(
415     X, y, test_size=0.2, random_state=42
416 )

```

```
417
418 # Scaling
419 scaler = StandardScaler()
420
421 X_train = scaler.fit_transform(X_train)
422 X_test = scaler.transform(X_test)
423
424 # Logistic Regression Model
425 model = LogisticRegression(max_iter=1000)
426
427 model.fit(X_train, y_train)
428
429 # Prediction
430 y_pred = model.predict(X_test)
431
432 # Accuracy
433 print("Accuracy :", accuracy_score(y_test, y_pred) * 100)
434
435
```